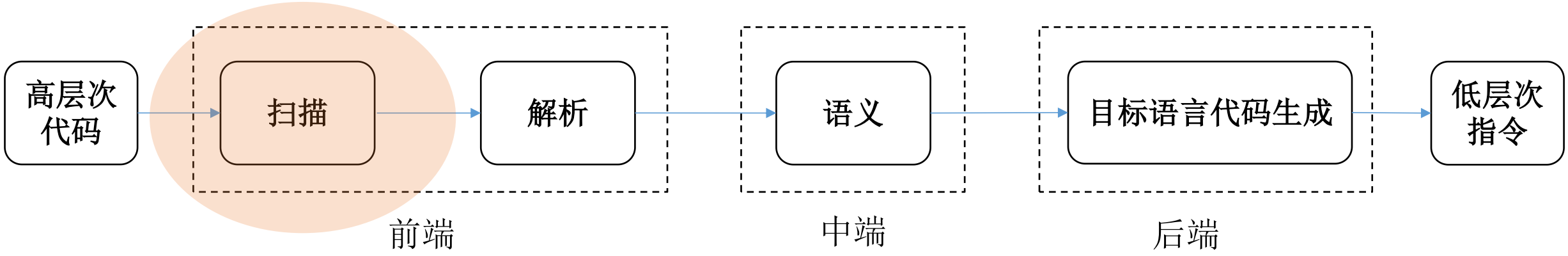
扫描和词法分析

杨广亮

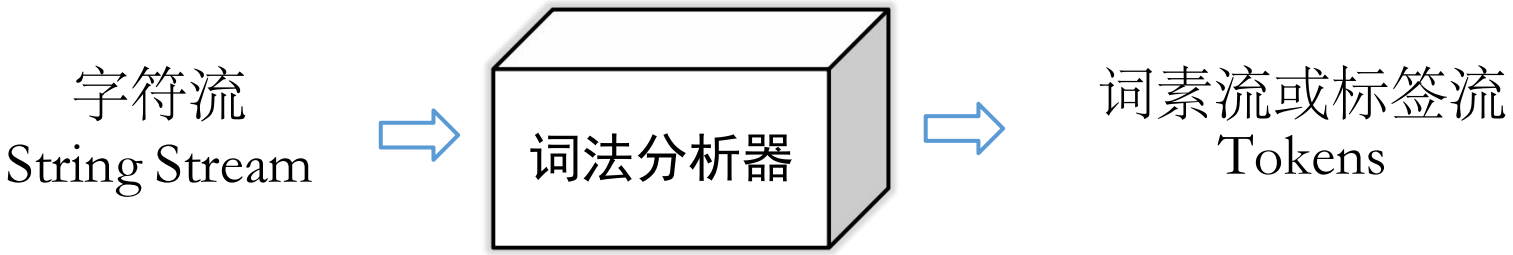
2026年3月


高层次代码
扫描
解析
语义
目标语言代码生成
低层次指令
前端
中端
后端

什么是扫描？

什么是扫描

- 什么是扫描?
  - 给定一段代码字符串，将其分解成基本元素，并对分解出的元素进行词性分析（例如判断是否为关键词，还是普通标识符）



什么是扫描

- 一个例子说明扫描的输入和输出

```
fn main() {
    let x = factorial(10);
    println("factorial(10) = {}", x);
}
```

什么是扫描

- 输入是字符串格式的源代码

IDE代码编辑器中的代码形式

```
fn main() {
    let x = factorial(10);
    println("factorial(10) = {}", x);
}
```

源代码 →

字符串格式下的“源代码”

```
fn main() {\n    let x = factorial(10);\nprintln("factorial(10) = {}", x);\n}
```

什么是扫描

- 输出是：划词分割后的内容（也叫词素）+类别（也叫标签），也叫Token

输入：字符串格式下的“源代码”

```
fn main() {\n    let x = factorial(10);\nprintln("factorial(10) = {}", x);\n}
```

扫描 →

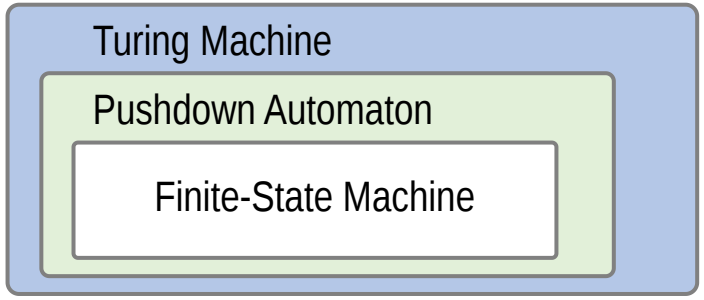
输出：Tokens（分割后的内容+类别）

```
<FN><IDENT,main><LPAR><RPAR><LBRACE><NL>
<LET><IDENT,x><EQ><IDENT,factorial><LPAR><IDENT>
<RPAR><SEMI><NL>
<IDENT,println><LPAR>
<STRING,"factorial(10) = {}">
<COMMA><IDENT,X><RPAR><SEMI><NL>
<RBRACE>
```

扫描原理：正则文法、正则表达式

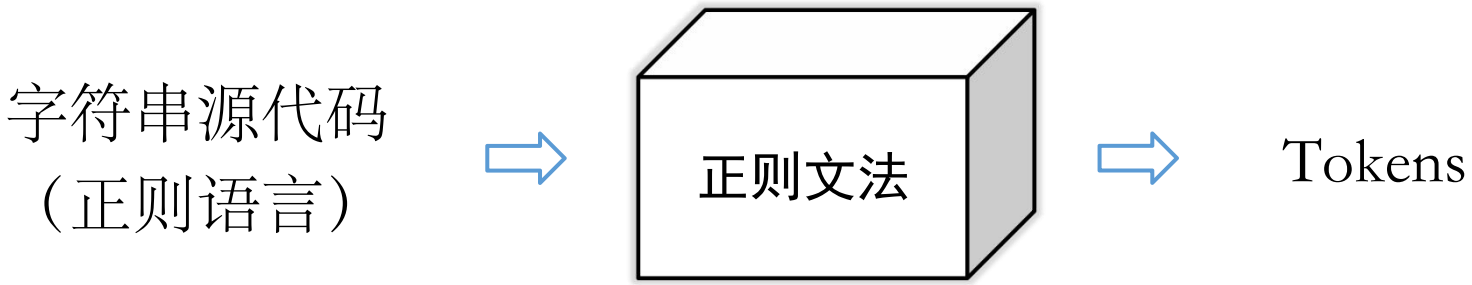
Chomsy语言体系

类型	文法名称	自动机模型	生成式形式	语言示例
0 型	递归枚举	图灵机	无限制	
1 型	上下文敏感	Linear bounded TM	左侧可以多个符号 $\alpha S \rightarrow \beta$	$a^n b^n c^n$
2 型	上下文无关	下推自动机	左侧仅一个符号$S \rightarrow \beta$	$a^n b^n$
3 型	正则	有穷自动机	右侧全部为终结符$S \rightarrow <a><b>$	a^n



正则语言或正则文法

- 正则文法是扫描的核心技术
- 正则文法：能被正则表达式接受的语言（字符串流）
- 正则表达式：一种（表达能力有限的）语言描述方法
  - 换句话说，可用正则表达式描述的语言称为正则语言



正则表达式：（1）对单一字符表达

- 表示字母表Σ上的字符串集合：
  - a：含义为$\{x|x = a\}$
  - $[ab]$：含义为$\{x|x = a \text{ or } x = b\}$
  - $[a-z]$：含义为$\{x|x = a \text{ or} \ldots \text{or } x = z\}$
  - $[a-zA-Z]$：含义为$\{x|x = a \text{ or} \ldots \text{or } x = z \text{ or} \ldots \text{or } x = Z\}$
  - $[\hat{}a]$：含义为$\{x|x! = a \text{ and } x \in \Sigma\}$
  - $a?$：含义为$\{x|x = a \text{ or } x = \epsilon\}$
  - .：通配符$\{x|x \in \Sigma\}$
  - ϵ：空
  - a|b：或

正则表达式：（2）对复合字符表达

- **选择**（choice）：$R|S$ ，含义为$\{x|x \in R \ or \ x \in S\}$
- **连接**（seq）：RS，含义为$\{x\,y|x \in R \ and \ \mathrm{y} \in S\}$
- **闭包**（Kleene closure / repeat）：R^*，含义为$\bigcup_{i=0}^{\infty} R^i$
  - 正闭包：R^+，含义为$\bigcup_{i=1}^{\infty} R^i$
  - 更多量化方式：{min, max}，如
    - a{1, 10}，表示任意1-10个a组成的字符串。
    - a{10}，表示10个a的组成的字符串。

正则表达式：基本运算法则

- 优先级顺序：
  - 闭包（$*$）优先级最高
  - 连接符其次
  - 选择符（|） 最低
- 运算法则：
  - 选择符满足交换律（commutative）：$r|s = s|r$，
  - 选择符满足结合律（associative）：$r|(s|t) = (r|s)|t$
  - 连接符满足结合律（associative）：$r(st) = (rs)t$
  - 连接符满足分配律（distributive）：$r(s|t) = rs|rt$
  - 闭包满足幂等率（idempotent）：$r^* = r^{**}$

正则集

- 一个正则表达式能够表示的语言就是正则集
- 假设$\Sigma = \{a, b\}$，则
  - $a|b$表示的语言为：{a,b}（称为正则集）
  - $(a|b)(a|b)$表示的语言为：{aa,ab,bb,ba}
  - a^*表示的语言为：$\{\epsilon$,a,aa,aaa,$\cdots\}$
  - $(a|b)^*$表示的语言为：$\{\epsilon$,a,b,aa,ab,ba,$\cdots\}$
  - $a|a^*b$表示的语言为：{a,aab,aaab,$\cdots$}

正则表达式等价性

- 如果两个正则表达式的正则集相等，则这两个正则表达式等价，如：
  - $a|b = b|a$
  - $(a|b)^* = (a^*|b^*)^*$
- 分析下列正则表达式是否等价?
  - $a^*(a|b)^*a$
  - $((\epsilon|a)b^*)^*$
  - $b^*(abb^*)^*)a|\epsilon($

扫描：如何用正则文法或表达式来分析计算机语言代码

- 基于该语言的规范和格式，针对设计正则文法
  - 在分析一门语言的时候，必须了解和定义这门语言的规范和格式的（即语法），包括什么是标识符、什么是关键字等等
- 语言的规范和格式
  - 数字：　[0-9]+ 或 $[0\text{-}9]^{+}(.[0\text{-}9]^{+}\,|\,\epsilon)$
  - 标识符：[_0-9a-zA-Z]+
  - 关键字列表

关键字列表：运算符和标签

二元运算符

<ADD> := +
<SUB> := -
<MUL> := *
<DIV> := /
<REM> := %

比较运算符

<CEQ> := ==
<NEQ> := !=
<GT> := >
<GTE> := >=
<LT> := <
<LTE> := <=

位运算符

<SHL> := <<
<SHR> := >>
<BAND> := &
<BOR> := |

赋值/一元运算符

<EQ> := =
<ADD> := ++
<SUB> := --
<MUL> := +=
<DIV> := -=

逻辑运算符

<AND> := &&
<OR> := ||
<NOT> := !

关键字列表：更多符号及其标签

域

<LPAR>	:=	(
<RPAR>	:=	)
<LSQ>	:=	[
<RSQ>	:=	]
<LBRA>	:=	{
<RBRA>	:=	}
<SQUT>	:=	'
<DQUT>	:=	"

其他

<COMMA>	:=	,	分隔多个元素
<SEMI>	:=	;	分隔多条语句
<RARROW>	:=	->	类型声明
<COLON>	:=	:	函数声明
<DOT>	:=	.	访问结构体内部
<DOTS>	:=	..	范围
<QUE>	:=	?	三元运算
<POUND>	:=	#	属性

注释

<SLASHES>	:=	//
<LSTAR>	:=	/*
<RSTAR>	:=	*/

关键字列表：保留字

函数、变量声明

<FN>	: =	fn
<LET>	: =	let

类型

<BOOL>	: =	bool
<CHAR>	: =	char
<INT>	: =	int
<LONG>	: =	long
<FLOAT>	: =	float
<DOUBLE>	: =	double
<STR>	: =	str
<STRUCT>	: =	struct

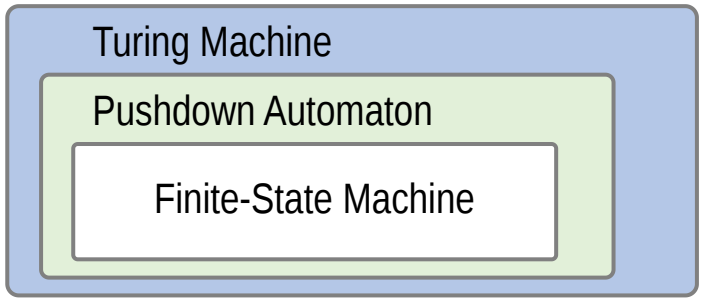
控制流

<IF>	: =	if
<ELSE>	: =	else
<FOR>	: =	for
<IN>	: =	in
<WHILE>	: =	while
<MATCH>	: =	match
<CASE>	: =	=>
<DEFAULT>	: =	_
<BREAK>	: =	break

正则文法底层实现：自动机

语言分析问题难度

类型	文法名称	自动机模型	生成式形式	语言示例
0 型	递归枚举	图灵机	无限制	
1 型	上下文敏感	Linear bounded TM	左侧可以多个符号 $\alpha S \rightarrow \beta$	$a^n b^n c^n$
2 型	上下文无关	下推自动机	左侧仅一个符号$S \rightarrow \beta$	$a^n b^n$
3 型	正则	有穷自动机	右侧全部为终结符$S \rightarrow$ <a><b>	a^n



有穷自动机（Deterministic Finite Automaton）

- 有穷自动机如果一个语言可以被有穷自动机识别则称之为正则语言，其定义为一个五元组

定义：

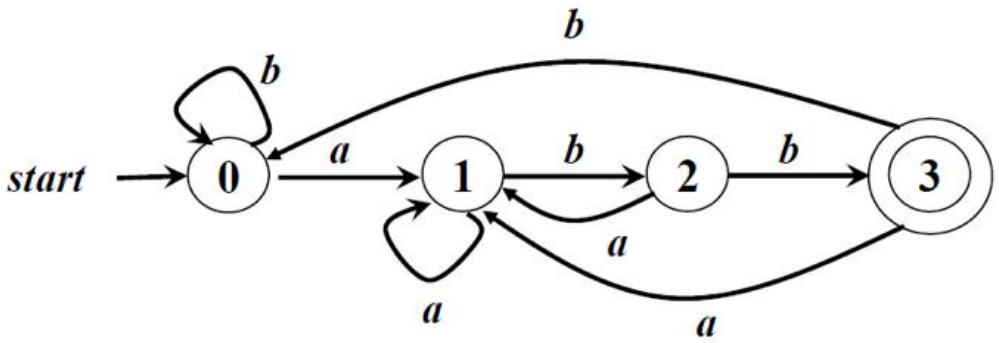
$$DFA = (Q, \Sigma, q_0, F, \delta)$$

说明：

1. Q : 有穷集合，状态集
2. Σ : 有穷集合，字母集
3. q_0 : $q_0 \in Q$, 表示开始/起始状态 (start/initial state)
4. F : $F \subseteq Q$, 表示最终/接受状态 (final/accept state)
5. $\delta : Q_1 \times \Sigma \rightarrow Q_2$ 转移函数

有穷自动机（Deterministic Finite Automaton）

$$M = (S, \Sigma, \delta, s_0, F)$$



转换表

状态 \ 输入	a	b
0	1	0
1	1	2
2	1	3
3	1	0

可以用转换表表示DFA

扫描

- 正则文法

正则语言 =>

等价

- 有穷自动机

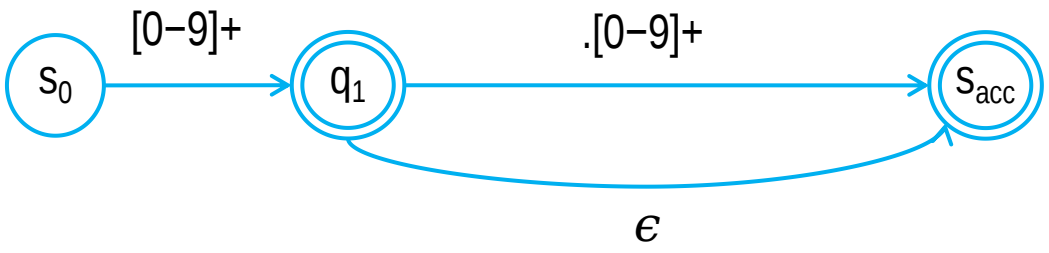
如何将正则表达式转换为FA?

- 如何构造正则表达式 $[0\text{-}9]^+((.[0\text{-}9]^+)\,|\,\epsilon)$对应的FA?

  - 识别无符号浮点数的FSA：
    - 字符集：$\sum = \{0,1,2,3,4,5,6,7,8,9,.\}$
    - 状态集：Q/$S = \{s_0,\ s_1,\ s_2,\ s_3\}$
    - 初始状态：$q_0/S_0 = s_0$
    - 接受状态：F/$S_{acc} = \{s_1, s_3\}$
    - 状态转移关系：$\Delta = \left\{ \begin{array}{c} s_0 \xrightarrow{[0-9]} s_1,\ s_1 \xrightarrow{[0-9]} s_1,\ s_1 \xrightarrow{.} s_2 \\ s_2 \xrightarrow{[0-9]} s_3,\ s_3 \xrightarrow{[0-9]} s_3 \end{array} \right\}$

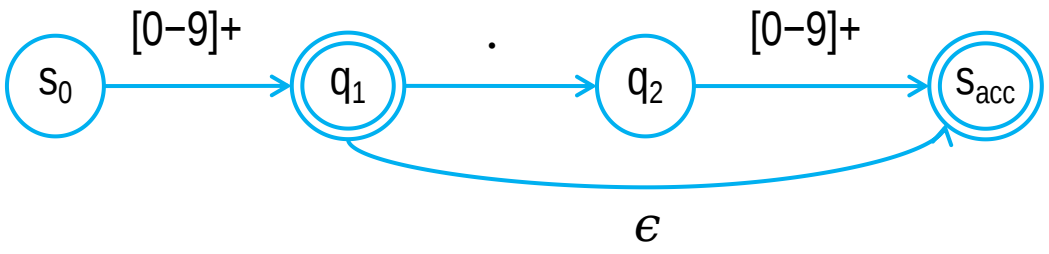
如何将正则表达式转换为FA?

- 如何构造正则表达式 $[0\text{-}9]^+((.[0\text{-}9]^+)\,|\,\epsilon)$对应的FA?



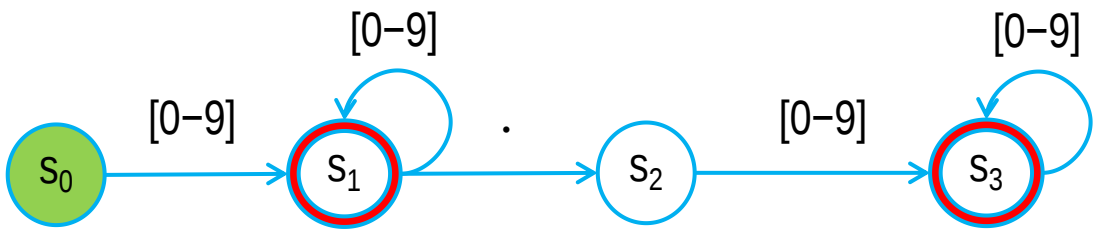
如何将正则表达式转换为FA?

- 如何构造正则表达式 $[0\text{-}9]^+((.[0\text{-}9]^+)\,|\,\epsilon)$对应的FA?



如何将正则表达式转换为FA?

- 如何构造正则表达式 $[0\text{-}9]^+((.[0\text{-}9]^+)\,|\,\epsilon)$对应的FA?



FSA接受字符串的条件

- 假设$\sum$ *是所有由属于$\sum$的元素组成的有限长度序列的集合(包含空字符串ϵ)，如1.23，
- FSA接受字符串$w = x_1 x_2 \ldots x_k \in \Sigma^*$的充要条件是:
  - 存在序列$s_{t_0} s_{t_1} \ldots s_{t_n} \in S$，其中$s_{t_0}$是初始状态，$s_{t_n} \in S_{acc}$
  - 并且$\forall s_{t_{i-1}}, x_i, s_{t_i}, (s_{t_{i-1}}, x_i, s_{t_i}) \in \Delta$
  - 即$\delta(\ldots\delta(\delta(s_{t_0}, x_1), x_2)\ldots, x_n) \in S_{acc}$
- FSA拒绝字符串的充要条件是:
  - 在某一状态（$s_{t_i} \notin S_{acc}$ ）无匹配的状态转移规则
  - 转移至拒绝状态s_{rej}

DFA和NFA等价

- 确定型有穷自动机（Deterministic FSA）
  - 对于FSA的一个状态和输入字符，最多只有一条状态转移边
- 非确定型有穷自动机（Nondeterministic FSA）
  - 对于FSA的一个状态和输入字符，可能存在多条状态转移边（ϵ转移）

定义：

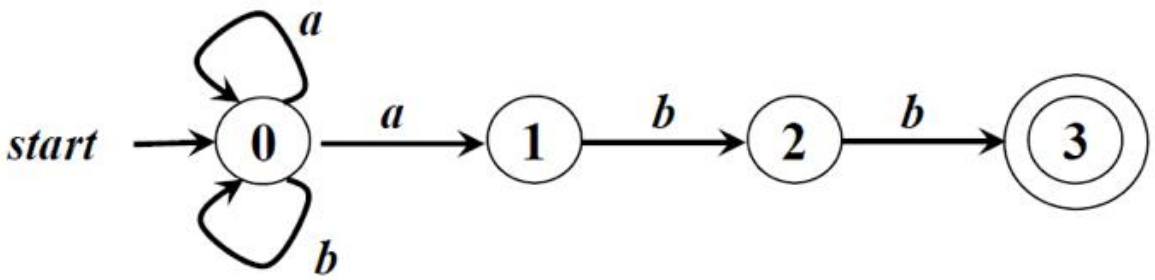
$$NFA = (Q, \Sigma, q_0, F, \delta)$$

说明：

1. 前四个与DFA定义相同
2. $\delta : Q_1 \times \Sigma \rightarrow P(Q)$ $P(Q)$表示一个状态的子集

非确定型有穷自动机

$$M = (S, \Sigma, \delta, s_0, F)$$



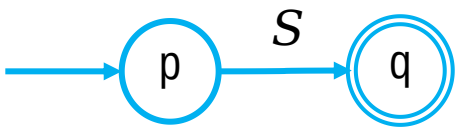
转换表

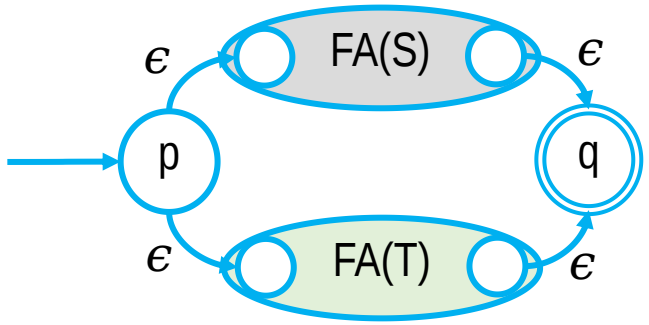
状态 \ 输入	a	b
0	{0,1}	{0}
1	Ø	{2}
2	Ø	{3}
3	Ø	Ø

如果转换函数没有给出对应于某个状态-输入对的信息，就把Ø放入相应的表项中

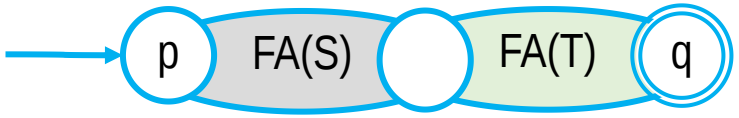
正则表达式转换为NFA

- Thompson构造法：McNaughton – Yamada – Thompson
- 将正则表达式递归展开为子表达式（只有一个符号）
  - 语法解析树
- 构造表达式的NFA
  - 选择：$S|T$
  - 连接：ST
  - 闭包：$S*$

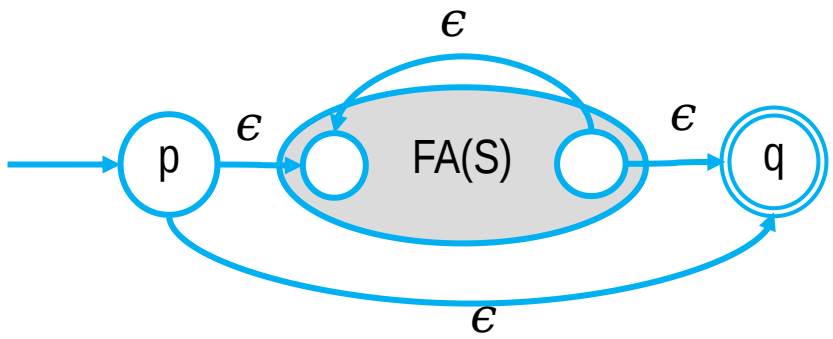




选择



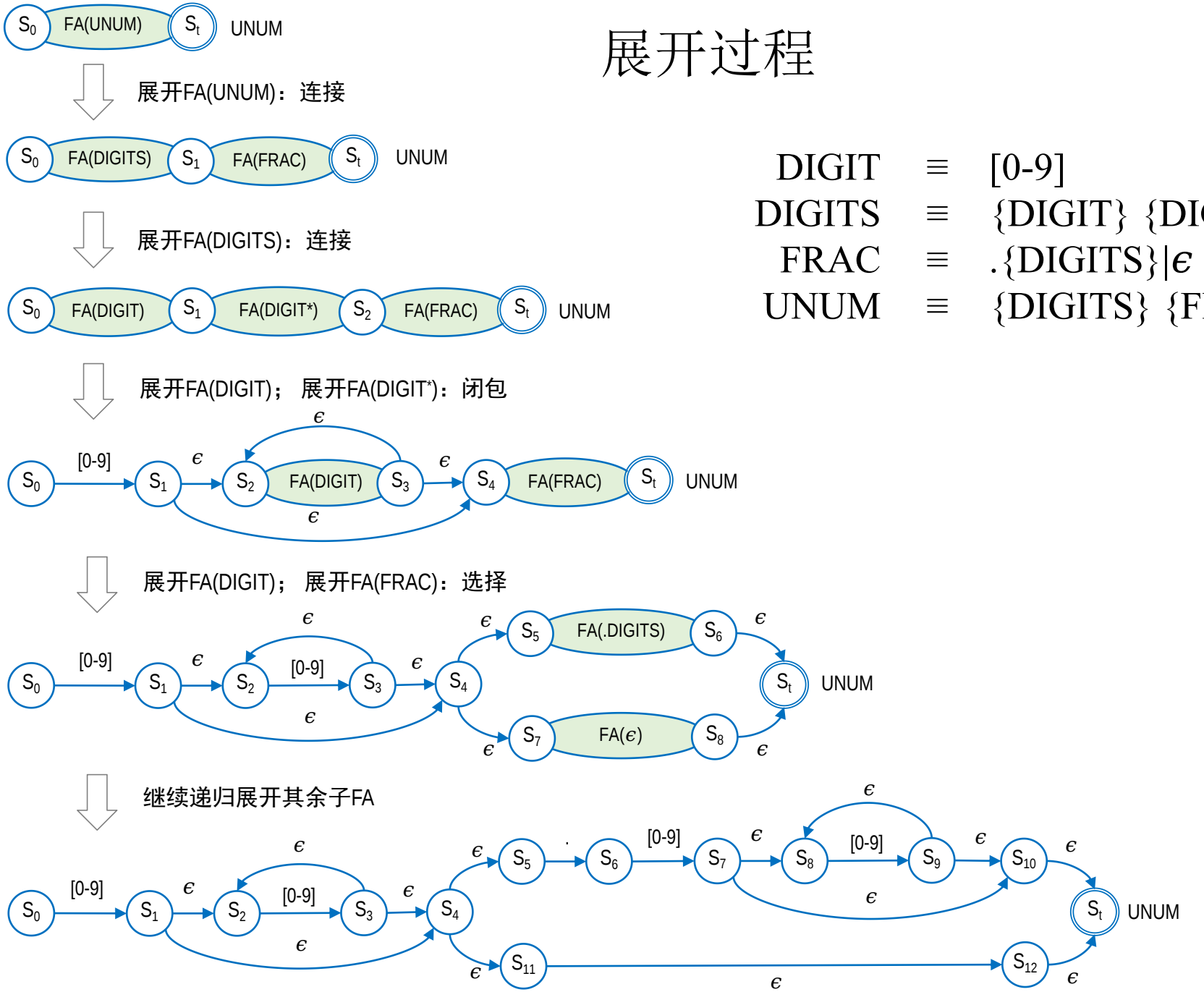
连接



重复

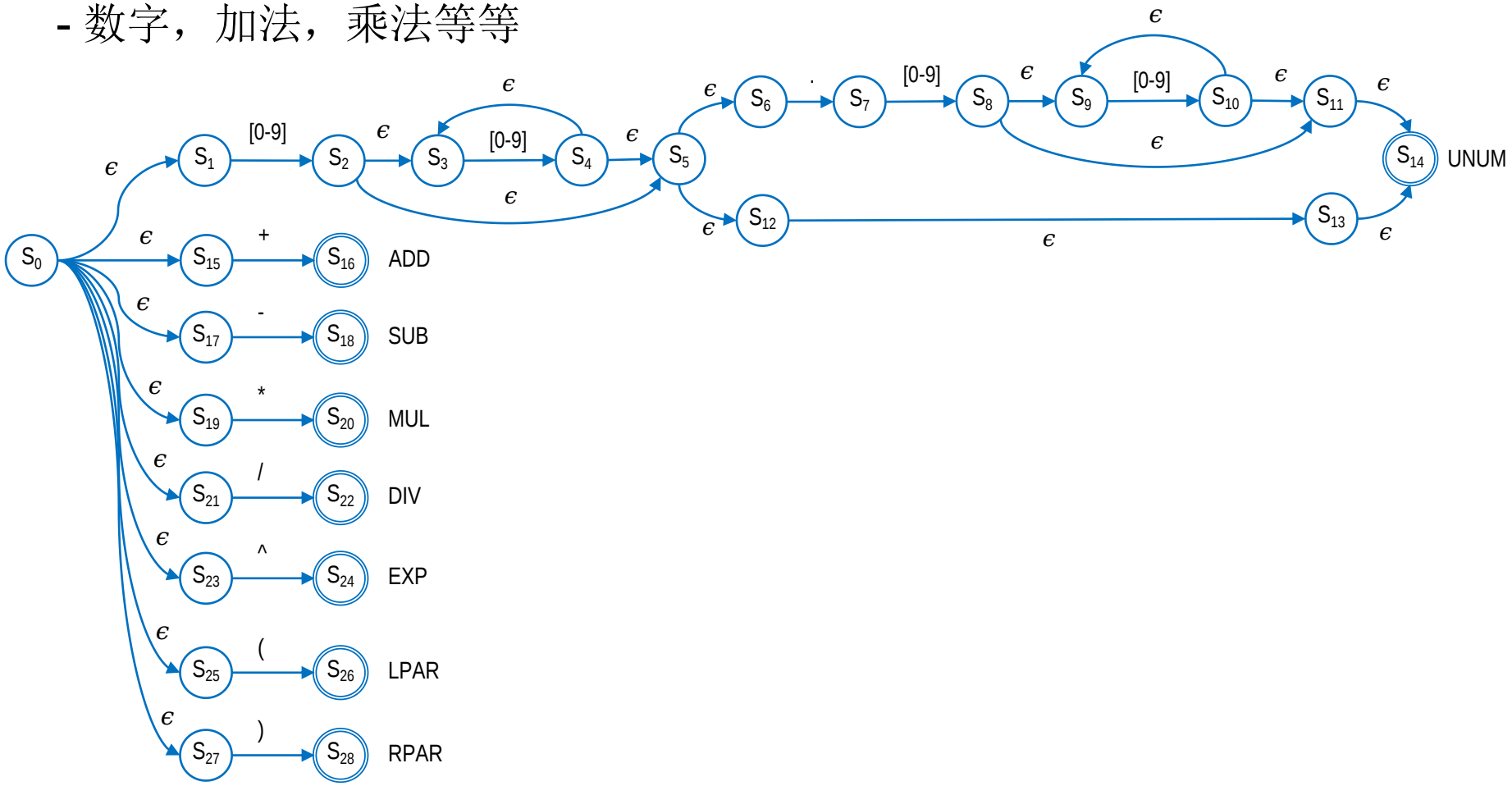
展开过程

$$
\begin{aligned}
\text{DIGIT} &\equiv [0\text{-}9] \\
\text{DIGITS} &\equiv \{\text{DIGIT}\}\ \{\text{DIGIT}\}^* \\
\text{FRAC} &\equiv .\{\text{DIGITS}\}|\epsilon \\
\text{UNUM} &\equiv \{\text{DIGITS}\}\ \{\text{FRAC}\}
\end{aligned}
$$



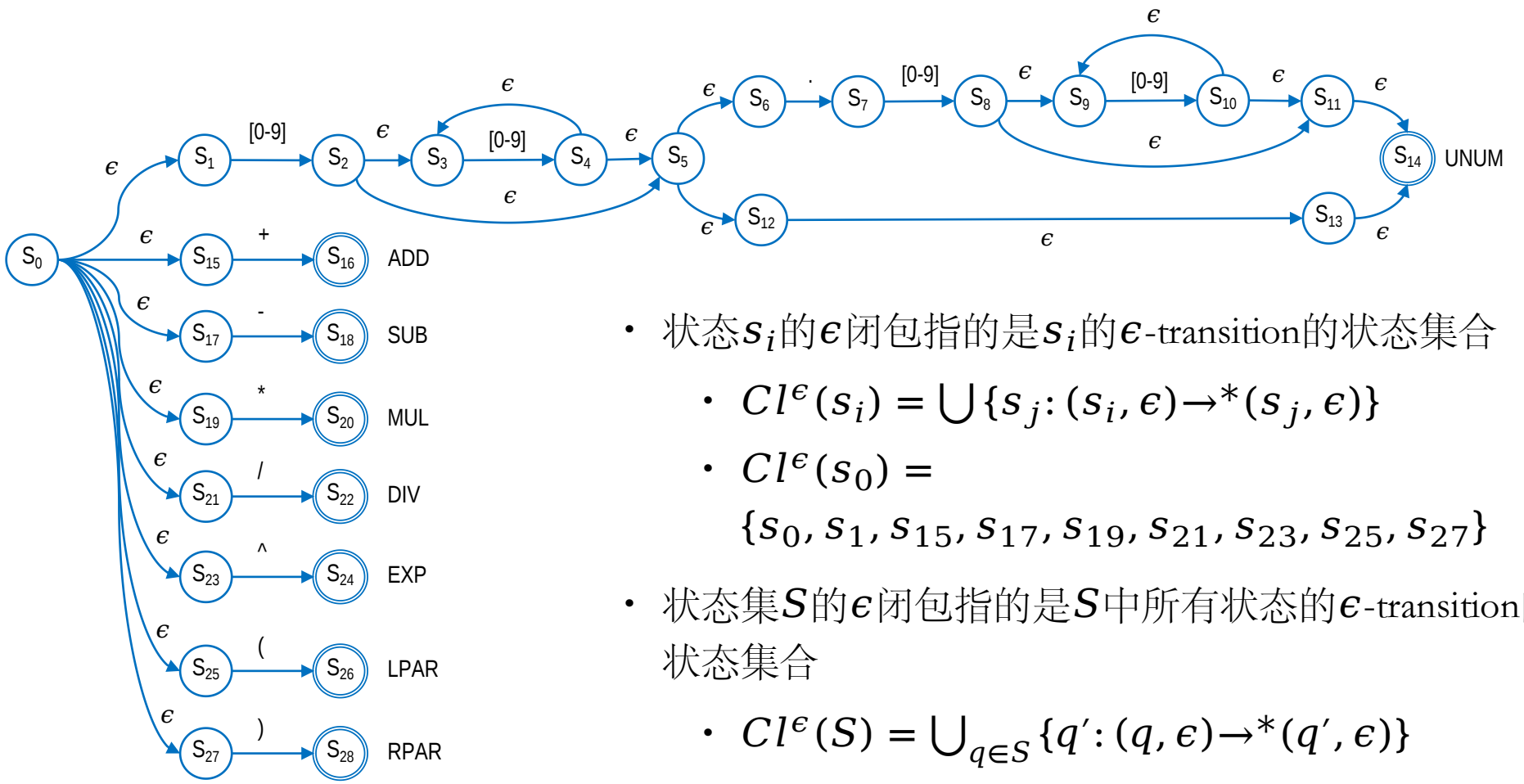
多个NFA合并

- 利用ϵ转移可以将多个NFA合并来表示多个正则表达式
  - 数字，加法，乘法等等



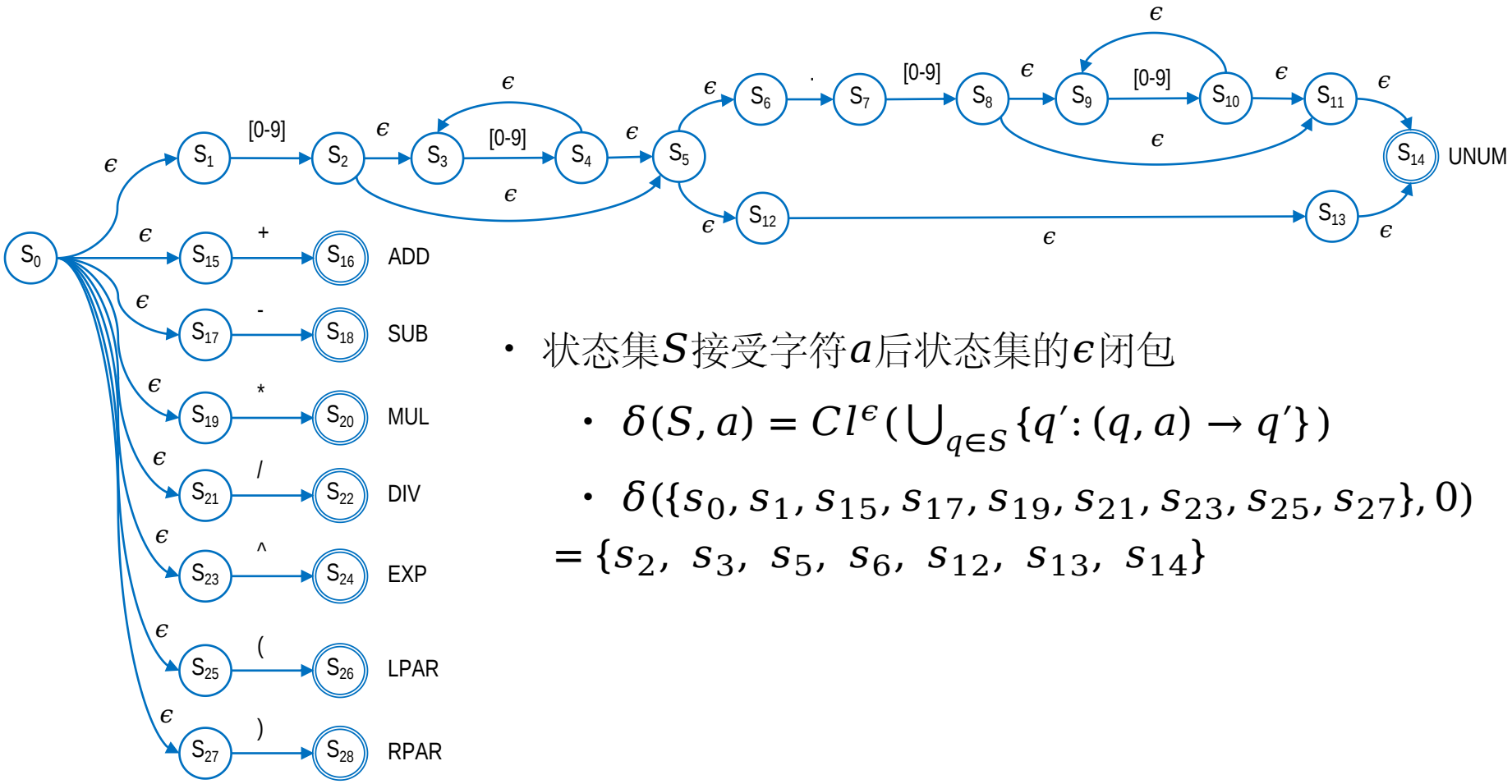
NFA不容易实现，是否可将其转换为DFA？

ϵ闭包（closure）



- 状态s_i的ϵ闭包指的是s_i的ϵ-transition的状态集合
  - $Cl^{\epsilon}(s_i) = \bigcup\{s_j : (s_i, \epsilon) \rightarrow^* (s_j, \epsilon)\}$
  - $Cl^{\epsilon}(s_0) =$ $\{s_0, s_1, s_{15}, s_{17}, s_{19}, s_{21}, s_{23}, s_{25}, s_{27}\}$
- 状态集S的ϵ闭包指的是S中所有状态的ϵ-transition的状态集合
  - $Cl^{\epsilon}(S) = \bigcup_{q \in S}\{q' : (q, \epsilon) \rightarrow^* (q', \epsilon)\}$

a-transition



- 状态集S接受字符a后状态集的ϵ闭包
  - $\delta(S,a) = Cl^{\epsilon}(\bigcup_{q\in S}\{q' : (q,a) \to q'\})$
  - $\delta(\{s_0, s_1, s_{15}, s_{17}, s_{19}, s_{21}, s_{23}, s_{25}, s_{27}\}, 0)$

$= \{s_2,\ s_3,\ s_5,\ s_6,\ s_{12},\ s_{13},\ s_{14}\}$

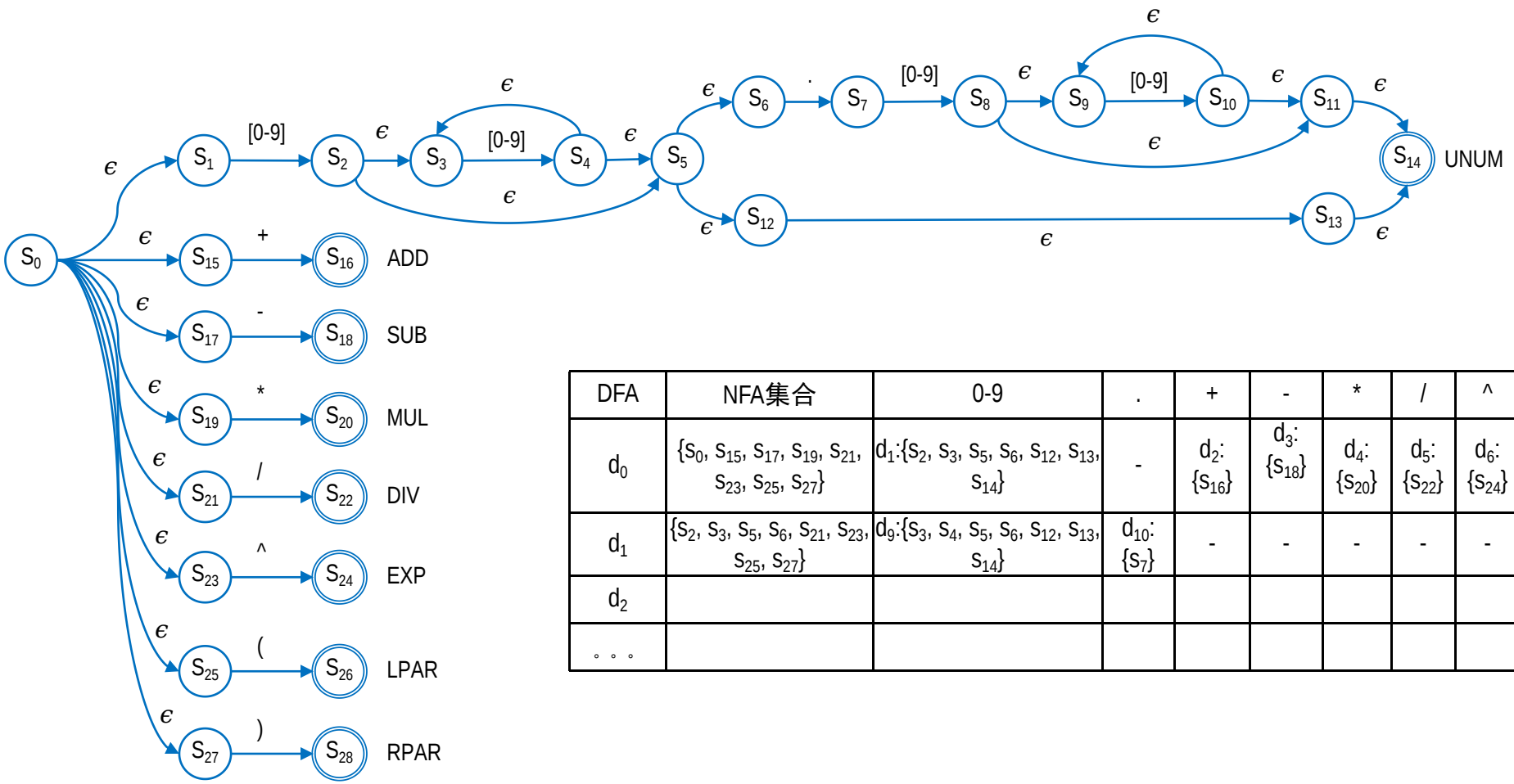
NFA转换为DFA：子集构造法

Powerset Construction

- 给定一个字符集Σ上的NFA $(N, \Delta, n_0, N_{acc})$，它对应的可接受同一语言的DFA $(D, \Delta', d_0, D_{acc})$定义如下：
  - D中的所有状态d_i都是N的一个子集，$D \subseteq 2^N$
  - $d_0 = Cl^{\epsilon}(n_0)$ //d_i均为ϵ闭包
  - $\Delta' = \{d_i \times c \times d_j\},\ \forall n_j \in d_j, \exists n_i \in d_i\ \&\ c \in \Sigma,\ \text{s.t.}\ (n_i, c, n_j) \in \Delta\}$
  - $D_{acc} = \{d_i \subseteq D \mid d_i \cap N_{acc} \neq \emptyset\}$

```
d0 = eclosure(n0);
D = d0; //保存得到的状态
worklist ={d0}; //待检验的状态
While (worklist!=null) do:
  worklist.remove(d);
  for each c in alphabets do:
    t = trans(d,c)
    if D.find(t) = null then:
      worklist.add(t);
      D.add(t);
```

构造过程

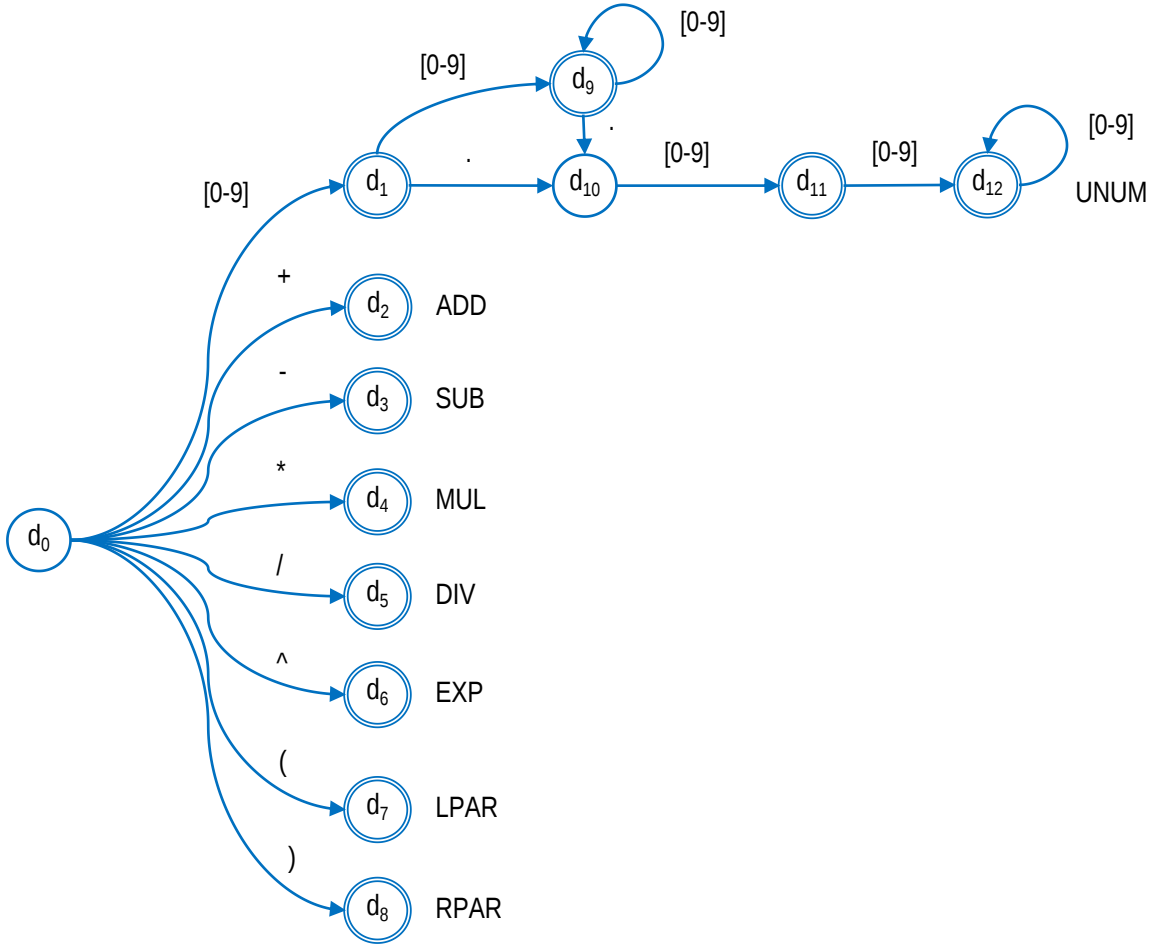


DFA	NFA集合	0-9	.	+	-	*	/	^	(	)
d_0	$\{S_0, S_{15}, S_{17}, S_{19}, S_{21}, S_{23}, S_{25}, S_{27}\}$	d_1:$\{S_2, S_3, S_5, S_6, S_{12}, S_{13}, S_{14}\}$	-	d_2: $\{S_{16}\}$	d_3: $\{S_{18}\}$	d_4: $\{S_{20}\}$	d_5: $\{S_{22}\}$	d_6: $\{S_{24}\}$	d_7: $\{S_{26}\}$	d_8: $\{S_{28}\}$
d_1	$\{S_2, S_3, S_5, S_6, S_{21}, S_{23}, S_{25}, S_{27}\}$	d_9:$\{S_3, S_4, S_5, S_6, S_{12}, S_{13}, S_{14}\}$	d_{10}: $\{S_7\}$	-	-	-	-	-	-	-
d_2										
。。。										

结果

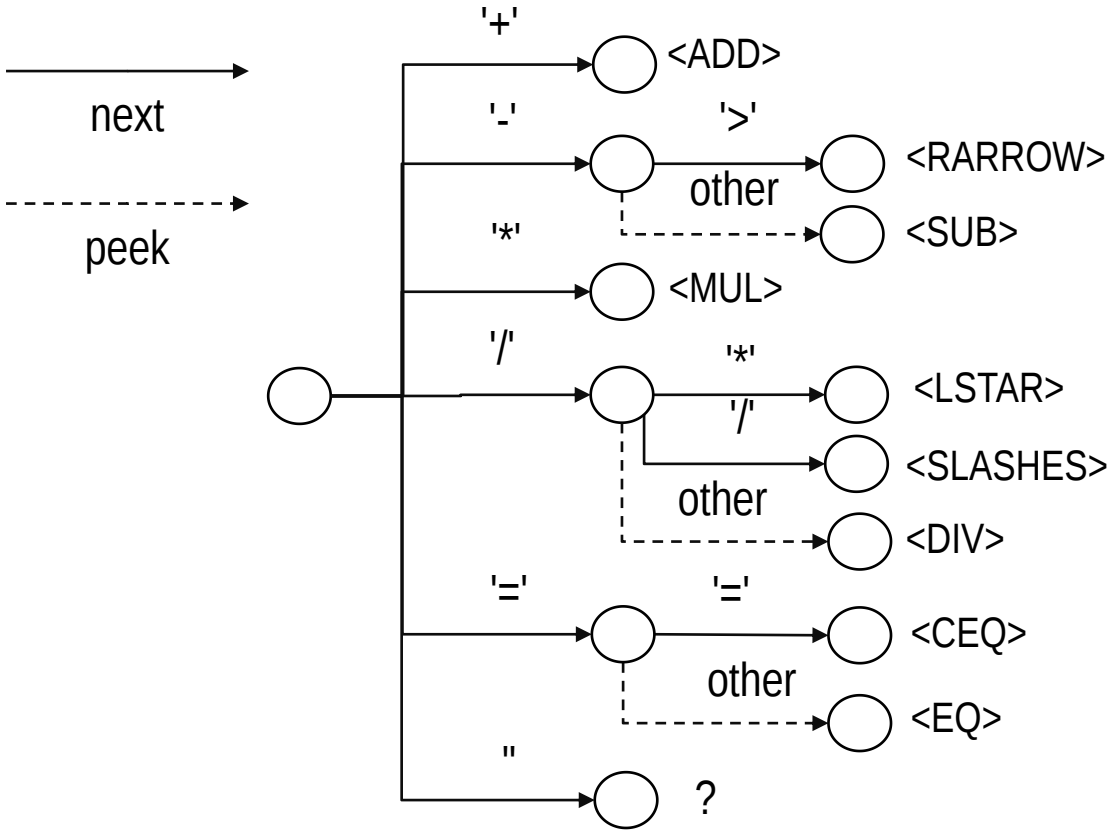
DFA 状态	NFA 状态集合	0-9	.	+	-	*	/	^	(	)
d_0	$\{s_0, s_{15}, s_{17}, s_{19}, s_{21}, s_{23}, s_{25}, s_{27}\}$	d_1: $\{s_2, s_3, s_5, s_6, s_{12}, s_{13}, s_{14}\}$	-	d_2: $\{s_{16}\}$	d_3: $\{s_{18}\}$	d_4: $\{s_{20}\}$	d_5: $\{s_{22}\}$	d_6: $\{s_{24}\}$	d_7: $\{s_{26}\}$	d_8: $\{s_{28}\}$
d_1	$\{s_2, s_3, s_5, s_6, s_{12}, s_{13}, s_{14}\}$	d_9: $\{s_3, s_4, s_5, s_6, s_{12}, s_{13}, s_{14}\}$	d_{10}: $\{s_7\}$	-	-	-	-	-	-	-
d_2	$\{s_{16}\}$	-	-	-	-	-	-	-	-	-
d_3	$\{s_{18}\}$	-	-	-	-	-	-	-	-	-
d_4	$\{s_{20}\}$	-	-	-	-	-	-	-	-	-
d_5	$\{s_{22}\}$	-	-	-	-	-	-	-	-	-
d_6	$\{s_{24}\}$	-	-	-	-	-	-	-	-	-
d_7	$\{s_{26}\}$	-	-	-	-	-	-	-	-	-
d_8	$\{s_{28}\}$	-	-	-	-	-	-	-	-	-
d_9	$\{s_3, s_4, s_5, s_6, s_{12}, s_{13}, s_{14}\}$	d_9	d_{10}	-	-	-	-	-	-	-
d_{10}	$\{s_7\}$	d_{11}: $\{s_8, s_9, s_{11}, s_{14}\}$	-	-	-	-	-	-	-	-
d_{11}	$\{s_8, s_9, s_{11}, s_{14}\}$	d_{12}: $\{s_9, s_{10}, s_{11}, s_{14}\}$	-	-	-	-	-	-	-	-
d_{12}	$\{s_9, s_10, s_{11}, s_{14}\}$	d_{12}	-	-	-	-	-	-	-	-

转换DFA结果


[0-9]
[0-9]
d9
.
[0-9]
[0-9]
[0-9]
d1
.
d10
d11
d12
UNUM
[0-9]
+
d2
ADD
-
d3
SUB
*
d4
MUL
d0
/
d5
DIV
^
d6
EXP
(
d7
LPAR
)
d8
RPAR

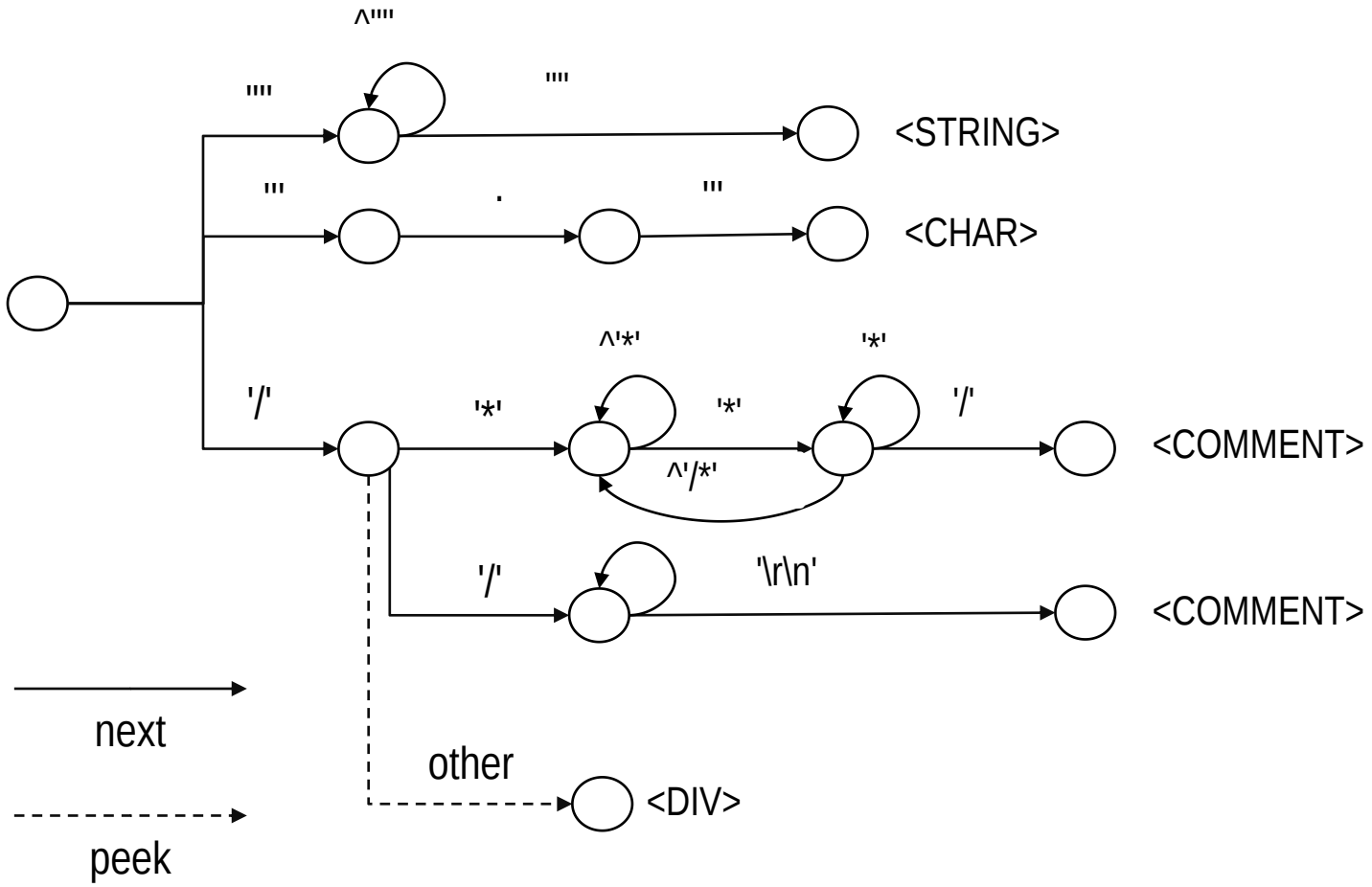
附录

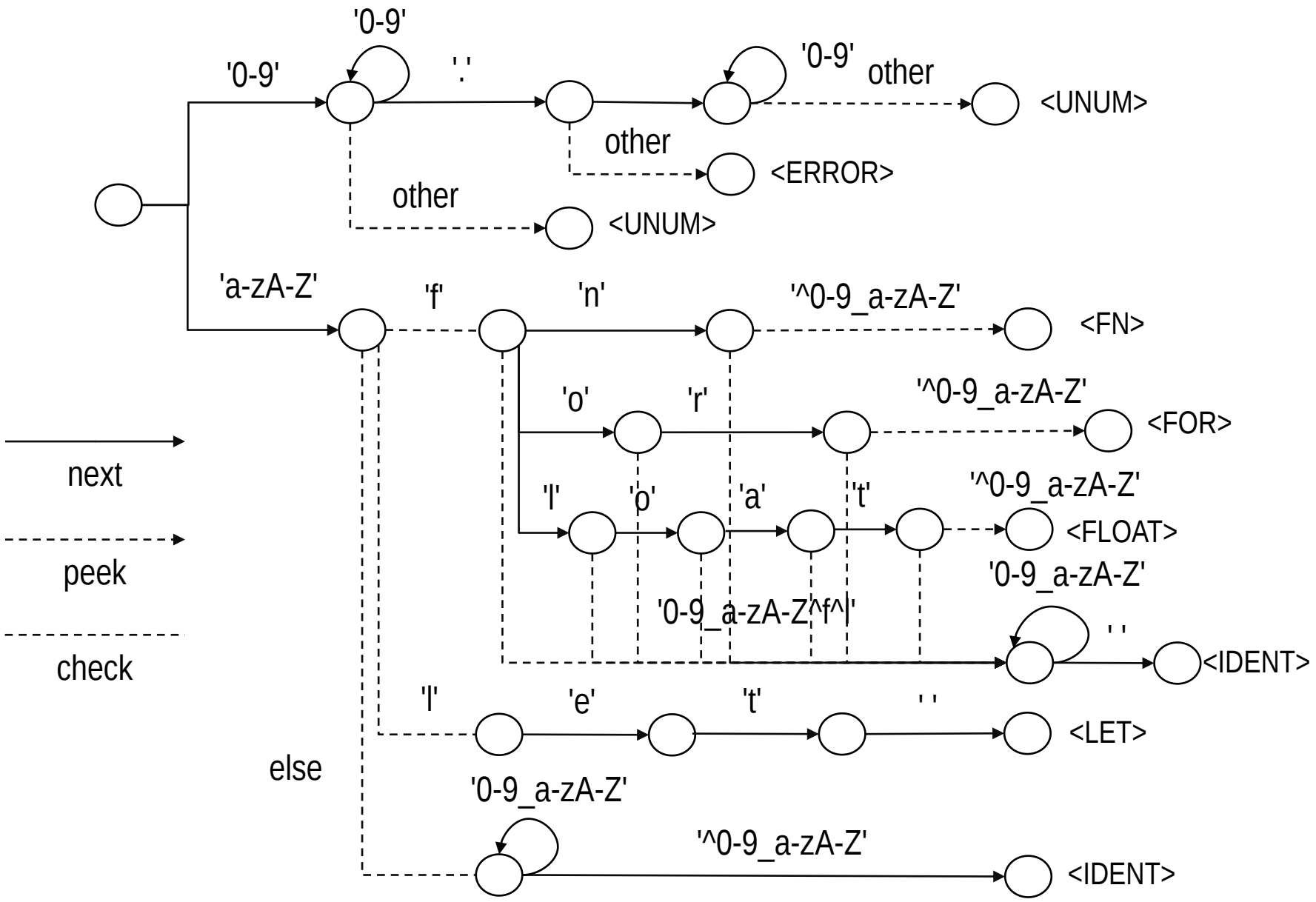
标签识别


next
peek
'+'
<ADD>
'-'
'>'
<RARROW>
other
<SUB>
'*'
<MUL>
'/'
'*'
<LSTAR>
'/'
<SLASHES>
other
<DIV>
'='
'='
<CEQ>
other
<EQ>
''
?

注译和引号

- 引号或注释内部的单词是否应识别为单独的标签？




'0-9'
'0-9'
'.'
'0-9'
other
<UNUM>
other
<ERROR>
other
<UNUM>
'a-zA-Z'
'f'
'n'
'^0-9_a-zA-Z'
<FN>
'o'
'r'
'^0-9_a-zA-Z'
<FOR>
'l'
'o'
'a'
't'
'^0-9_a-zA-Z'
<FLOAT>
'0-9_a-zA-Z'
'0-9_a-zA-Z^f^l'
' '
<IDENT>
'l'
'e'
't'
' '
<LET>
else
'0-9_a-zA-Z'
'^0-9_a-zA-Z'
<IDENT>
next
peek
check